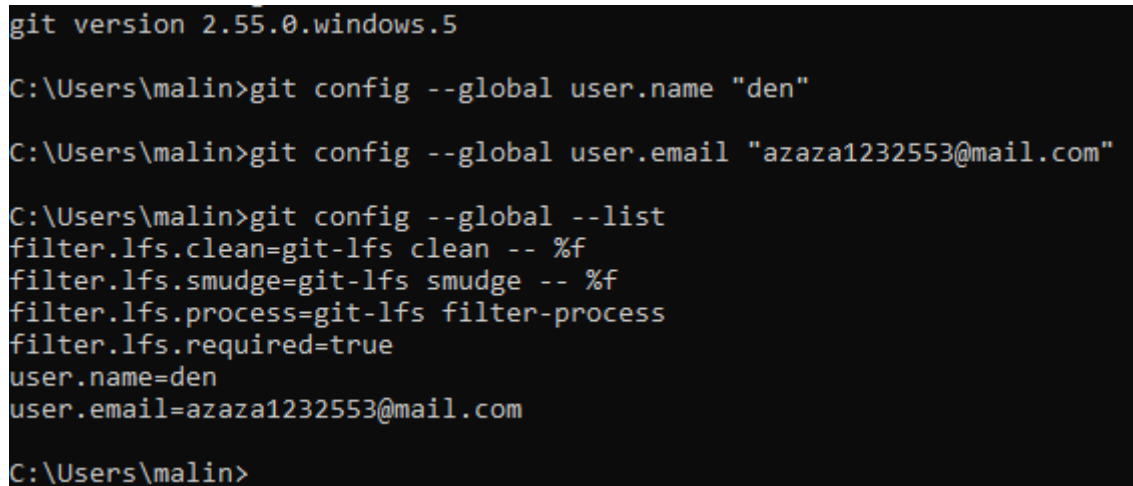


Вариативная самостоятельная работа №2.3

Выполнение наиболее распространённых команд в терминале Git

Команды будут выполнены в cmd.exe.

Для того что бы начать пользоваться git, необходимо ввести своё имя и адрес эл. почты.



```
git version 2.55.0.windows.5

C:\Users\malin>git config --global user.name "den"

C:\Users\malin>git config --global user.email "azaza1232553@mail.com"

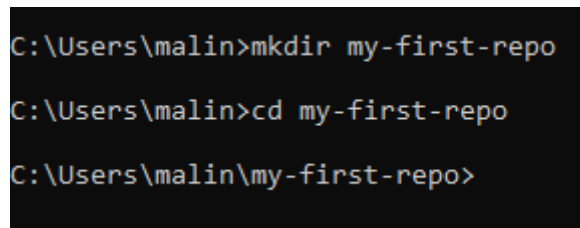
C:\Users\malin>git config --global --list
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
user.name=den
user.email=azaza1232553@mail.com

C:\Users\malin>
```

Рис. 1 – редактирование имени пользователя и его эл. почты

Здесь мы обращаемся к Git и с помощью метода config меняем конфигурацию, а флаг --global нужен для того что бы применить эту настройку ко всем репозиториям. User.email и user.name это параметры в которые мы заносим наше имя и эл. Почту.

Теперь после того как мы представились, можно создать новый репозиторий.



```
C:\Users\malin>mkdir my-first-repo

C:\Users\malin>cd my-first-repo

C:\Users\malin\my-first-repo>
```

Рис. 2 – создание папки для репозитория

Для этого воспользуемся методом mkdir, что бы создать папку. После создания перейдем в неё с помощью команды cd

После этого можно создавать репозиторий, для этого используем команду init.

```
C:\Users\malin\my-first-repo>git init
Initialized empty Git repository in C:/Users/malin/my-first-repo/.git/

C:\Users\malin\my-first-repo>_
```

Рис. 3 – создание репозитория

После создания репозитория мы можем загрузить туда необходимые нам файлы и «закоммитить» их.

```
C:\Users\malin\my-first-repo>echo Hello, Git! > readme.txt
C:\Users\malin\my-first-repo>
```

Рис. 4 – создание файла

В этой строке мы сначала вызываем метод вывода текста на экран, пишем текст и перенаправляем вывод в файл. После выполнения этой команды в нашем репозитории появится файл с текстом

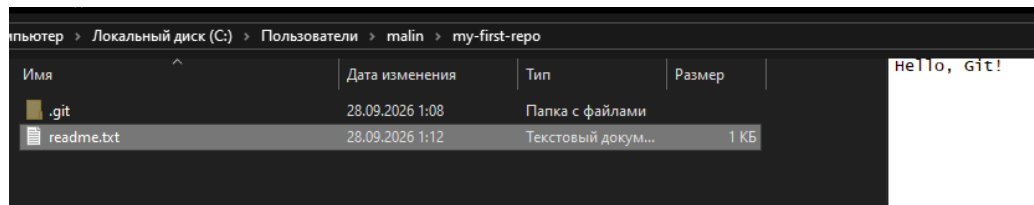


Рис. 5 – созданный файл

Так же мы можем проверить, видит ли Git наш файл, для этого выполняем команду представленную на рисунке ниже

```
C:\Users\malin\my-first-repo>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    readme.txt

nothing added to commit but untracked files present (use "git add" to track)
C:\Users\malin\my-first-repo>
```

Рис. 6 – результат выполнения команды

Здесь мы видим, что Git видит наш файл, но он ещё не отслеживается.

Для того что бы файл отслеживался мы можем выполнить команду, представленную ниже.

```
C:\Users\malin\my-first-repo>git add readme.txt

C:\Users\malin\my-first-repo>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.txt

C:\Users\malin\my-first-repo>
```

Рис. 7 – результат выполнения команды

Теперь мы видим, что Git помечает наш файл и видит его.

Теперь мы можем «закоммитить» все наши изменения, тем самым сохранить их.

```
C:\Users\malin\my-first-repo>git commit -m "first blood"
[master (root-commit) 6b1fb93] first blood
1 file changed, 1 insertion(+)
 create mode 100644 readme.txt

C:\Users\malin\my-first-repo>_
```

Рис. 8 – фиксация изменений

Что бы проверить сохранение можно посмотреть историю «коммитов».

```
C:\Users\malin\my-first-repo>git log --oneline
6b1fb93 (HEAD -> master) first blood

C:\Users\malin\my-first-repo>_
```

Рис. 9 – вызов истории изменений

Вывод в консоли позволяет нам понять, что последняя операция прошла успешно.

На этом моменте заканчивается набор базовых команд Git для cmd.

Далее будет произведена выгрузка хранимых на компьютере файлов в заранее созданный репозиторий gitlab. Для этого нам необходимо клонировать с сайта Gitlab репозиторий на наш компьютер, после чего перейти в перенесённую папку.

```
C:\Users\malin\my-first-repo>git clone https://git.herzen.spb.ru/1161516/eduprac-3.git
Cloning into 'eduprac-3'...
warning: missing OAuth configuration for git.herzen.spb.ru - see https://aka.ms/gcm/gitlab for more information
warning: You appear to have cloned an empty repository.

C:\Users\malin\my-first-repo>git clone https://git.herzen.spb.ru/1161516/eduprac-3.git
fatal: destination path 'eduprac-3' already exists and is not an empty directory.

C:\Users\malin\my-first-repo>cd eduprac-3

C:\Users\malin\my-first-repo\eduprac-3>
```

Рис. 10 – выполнение команд

Теперь, когда репозиторий с Gitlab находится на нашем компьютере, мы можем поместить в папку репозитория все необходимые для выгрузки в репозиторий на Gitlab файлы, после чего выгрузить их.

```
C:\Users\malin\my-first-repo\eduprac-3>dir
Том в устройстве C не имеет метки.
Серийный номер тома: 6AB4-326F

Содержимое папки C:\Users\malin\my-first-repo\eduprac-3

28.09.2026  02:24    <DIR>          .
28.09.2026  02:24    <DIR>          ..
28.09.2026  02:21             0 README.md
28.09.2026  02:24    <DIR>          BCP 1.1
28.09.2026  02:24    <DIR>          BCP 2.2
28.09.2026  02:24    <DIR>          BCP 2.3
28.09.2026  02:24    <DIR>          документы
28.09.2026  02:24    <DIR>          ИСР 1.1
28.09.2026  02:24    <DIR>          ИСР 1.2
28.09.2026  02:24    <DIR>          ИСР 1.3
                1 файлов             0 байт
                9 папок  80 955 293 696 байт свободно

C:\Users\malin\my-first-repo\eduprac-3>git status
On branch master
Your branch is up to date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    "\320\222\320\241\320\240 1.1/"
    "\320\222\320\241\320\240 2.2/"
    "\320\222\320\241\320\240 2.3/"
    "\320\230\320\241\320\240 1.1/"
    "\320\230\320\241\320\240 1.2/"
    "\320\230\320\241\320\240 1.3/"
    "\320\264\320\276\320\272\321\203\320\274\320\265\320\275\321\202\321\213/"

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\malin\my-first-repo\eduprac-3>git config --global core.quotepath false
```

Рис. 11 – добавленные файлы

Мы добавили файлы, теперь необходимо чтобы git начал их отслеживать.

```
C:\Users\malin\my-first-repo\eduprac-3>git add .

C:\Users\malin\my-first-repo\eduprac-3>git status
On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   BCP 1.1/BCP 2.1.docx
    new file:   BCP 1.1/BCP 2.1.pdf
    new file:   BCP 2.2/~$CP 1.1.docx
    new file:   BCP 2.2/BCP 2.2.docx
    new file:   BCP 2.2/BCP 2.2.pdf
    new file:   BCP 2.3/~$CP 1.1.docx
    new file:   BCP 2.3/~$CP 2.3.docx
    new file:   BCP 2.3/BCP 2.3.docx
    new file:   ИСР 1.1/ИСР 1.1.docx
    new file:   ИСР 1.1/ИСР 1.1.pdf
    new file:   ИСР 1.2/~$CP 1.1.docx
    new file:   ИСР 1.2/~WRL0005.tmp
    new file:   ИСР 1.2/ИСР 1.2.docx
    new file:   ИСР 1.2/ИСР 1.2.pdf
    new file:   ИСР 1.3/~$CP 1.1.docx
    new file:   ИСР 1.3/~WRL0005.tmp
    new file:   ИСР 1.3/ИСР 1.3.docx
    new file:   ИСР 1.3/ИСР 1.3.pdf
    new file:   документы/Титульный лист.docx
    new file:   документы/Титульный лист.pdf
    new file:   документы/задание_проектно-технологическая.docx
    new file:   документы/задание_проектно-технологическая.pdf
    new file:   документы/отчет_проектно-технологическая.docx

C:\Users\malin\my-first-repo\eduprac-3>_
```

Рис. 12 – добавленные файлы теперь отслеживаются

Теперь мы можем «закоммитить» добавленные файлы.

```
C:\Users\malin\my-first-repo\eduprac-3>git commit -m "Добавлены работы"
[master 23d87fb] Добавлены работы
23 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 BCP 1.1/BCP 2.1.docx
create mode 100644 BCP 1.1/BCP 2.1.pdf
create mode 100644 BCP 2.2/~$CP 1.1.docx
create mode 100644 BCP 2.2/BCP 2.2.docx
create mode 100644 BCP 2.2/BCP 2.2.pdf
create mode 100644 BCP 2.3/~$CP 1.1.docx
create mode 100644 BCP 2.3/~$CP 2.3.docx
create mode 100644 BCP 2.3/BCP 2.3.docx
create mode 100644 ИСП 1.1/ИСП 1.1.docx
create mode 100644 ИСП 1.1/ИСП 1.1.pdf
create mode 100644 ИСП 1.2/~$CP 1.1.docx
create mode 100644 ИСП 1.2/~WRL0005.tmp
create mode 100644 ИСП 1.2/ИСП 1.2.docx
create mode 100644 ИСП 1.2/ИСП 1.2.pdf
create mode 100644 ИСП 1.3/~$CP 1.1.docx
create mode 100644 ИСП 1.3/~WRL0005.tmp
create mode 100644 ИСП 1.3/ИСП 1.3.docx
create mode 100644 ИСП 1.3/ИСП 1.3.pdf
create mode 100644 документы/Титульный лист.docx
create mode 100644 документы/Титульный лист.pdf
create mode 100644 документы/задание_проектно-технологическая.docx
create mode 100644 документы/задание_проектно-технологическая.pdf
create mode 100644 документы/отчет_проектно-технологическая.docx

C:\Users\malin\my-first-repo\eduprac-3>
```

Рис. 13 – «КОММИТ»

Теперь что бы файлы оказались на Gitlab, необходимо выполнить команду, представленную на рисунке ниже.

```
C:\Users\malin\my-first-repo\eduprac-3>git push
Enumerating objects: 28, done.
Counting objects: 100% (28/28), done.
Delta compression using up to 12 threads
Compressing objects: 100% (27/27), done.
Writing objects: 100% (27/27), 1.39 MiB | 4.61 MiB/s, done.
Total 27 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
To https://git.herzen.spb.ru/1161516/eduprac-3.git
   05e4ecc..23d87fb  master -> master

C:\Users\malin\my-first-repo\eduprac-3>_
```

Рис. 14 – выгрузка файлов в Gitlab

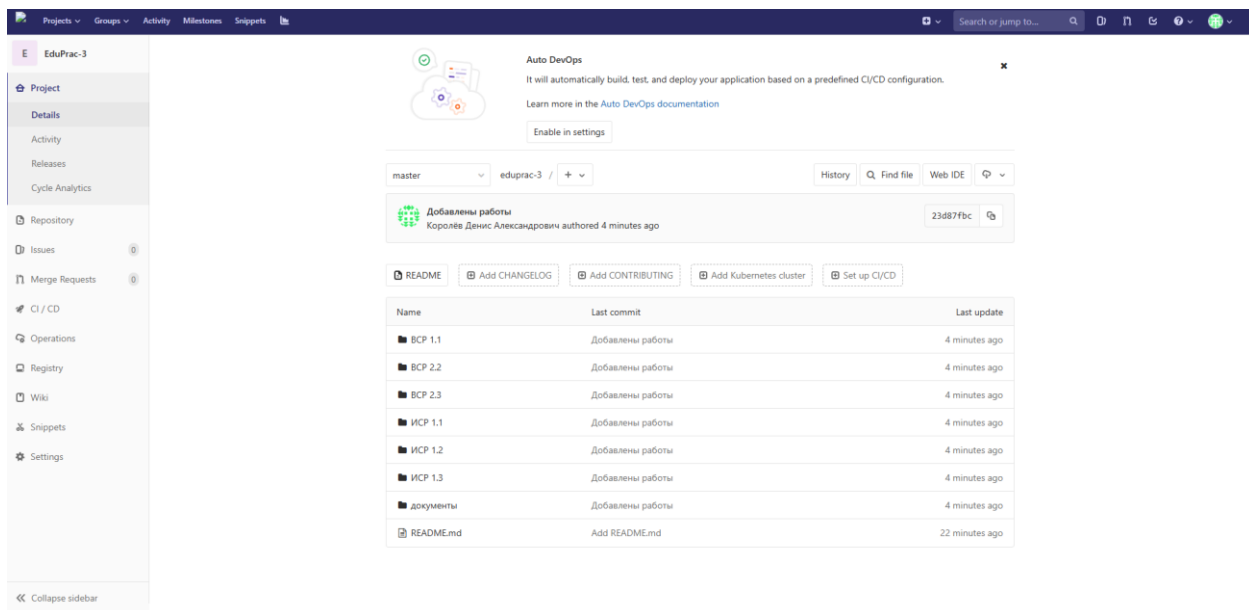


Рис. 15 – файлы в Gitlab

Все использованные переменные, методы и параметры описаны в списке, представленном ниже:

- *git --version* — проверяет, установлен ли Git в системе, и выводит его версию. У меня показало *git version 2.55.0.windows.5*, значит Git установлен корректно и доступен из командной строки.
- *git config --global user.name "Имя"* — задаёт имя пользователя, которое будет записываться в каждый коммит. Ключ *--global* означает, что настройка применяется ко всем репозиториям на компьютере, а не только к текущему.
- *git config --global user.email "email"* — задаёт email пользователя для коммитов. Нужен, чтобы в истории было видно, кто внёс изменения.
- *git config --global --list* — выводит список всех глобальных настроек. Я использовал её для проверки, что имя и email записались правильно.
- *git config --global core.quotePath false* — отключает escape-кодирование имён файлов. Без этой настройки Git показывает русские имена папок в виде *\320\222\320\241\320\240*, (см. рис. 11). Это важно для работы с кириллицей.
- *git init* — инициализирует Git-репозиторий в текущей папке. Создаётся скрытая папка *.git*, в которой будет храниться вся история изменений. С этого момента папка становится репозиторием.
- *git clone <URL>* — создаёт локальную копию удалённого репозитория: скачивает все файлы и всю историю коммитов. Автоматически настраивает связь с удалённым репозиторием под именем *origin*. Я использовал это для получения репозитория *eduprac-3* с GitLab.
- *git status* — показывает текущее состояние репозитория: какие файлы изменены, какие добавлены в индекс, какие ещё не отслеживаются. Я

запускал её несколько раз, чтобы убедиться, что файлы попали в индекс перед коммитом.

- *git add <файл>* — добавляет указанный файл в индекс (staging area) — «черновик» будущего коммита.
- *git add .* — добавляет **все** файлы и папки из текущей директории в индекс. Точка означает «текущая папка». Я использовал эту команду, чтобы подготовить все учебные работы к коммиту.
- *git commit -m "Сообщение"* — сохраняет все подготовленные (через *git add*) изменения в историю репозитория. Ключ *-m* позволяет указать текст сообщения прямо в команде. Я использовал это дважды: для тестового файла ("first blood") и для учебных работ ("Добавлены работы").
- *git log --oneline* — показывает историю коммитов в кратком виде: короткий хеш и сообщение коммита. Ключ *--oneline* делает вывод компактным. Я использовал её для проверки, что коммит создался.
- *git push* — отправляет локальные коммиты на удалённый репозиторий. Поскольку репозиторий был получен через *git clone*, Git уже знает адрес (origin) и имя ветки (master), поэтому дополнительные параметры не нужны. После этой команды все мои файлы появились на GitLab.
- *mkdir <имя>* — создаёт папку с указанным именем.
- *cd <путь>* — переходит в указанную папку. Change Directory.
- *cd ..* — переходит на одну папку выше (в родительскую).
- *dir* — показывает список файлов и папок в текущей директории. Использовал для проверки, что файлы скопировались.
- *echo Текст > файл.txt* — создаёт файл с указанным текстом. Знак *>* перенаправляет вывод команды *echo* в файл вместо экрана.